

Báo Cáo Dự Án: Phần 1

Tên: Lê Phạm Đăng Khiêm

MSSV: 24120341

Môn: Toán ứng dụng và thống kê

Mục lục

1	Giới Thiệu Dự Án	3
2	Tổng Quan Cấu Trúc Dự Án	3
2.1	Danh Sách File	3
3	Thuật Toán Khử Gauss (gaussian.py)	3
3.1	Mô Tả Chung	3
3.1.1	Các Hàm Hỗ Trợ	3
3.1.2	Khử Ma Trận về Bậc Thang	4
3.1.3	Tìm Nghiệm Hệ Phương Trình	4
3.1.4	Cách Sử Dụng	4
4	Tính Định Thức (determinant.py)	6
4.1	Thuật Toán	6
4.2	Cách Sử Dụng	6
4.3	Các Trường Hợp Kiểm Thử	6
5	Tính Ma Trận Nghịch Đảo (inverse.py)	7
5.1	Thuật Toán Gauss-Jordan	7
5.2	Cách Sử Dụng	7
5.3	Ứng Dụng Kiểm Thử	7
6	Tính Hạng và Cơ Sở (rank_basis.py)	8
6.1	Định Nghĩa	8
6.2	Thuật Toán	8
6.3	Cách Sử Dụng	8
7	Kiểm Chứng và Xác Nhận (part1_demo.ipynb)	9
7.1	Mục Đích	9
7.2	Nội Dung Chính	9
7.2.1	Test Gaussian Elimination	9
7.2.2	Test Back Substitution	9
7.2.3	Test Determinant	9
7.2.4	Test Matrix Inverse	9
7.2.5	Test Rank and Basis	10
7.3	Kiểm Chứng bằng NumPy	10

8	Hướng Dẫn Sử Dụng (README.md)	11
8.1	Giải Thích Hàm gaussian_eliminate	11
8.1.1	Ý Nghĩa của Nghiệm Hệ	11
8.1.2	Ví Dụ Kiểm Tra Nhanh	11
8.2	Chuyển Đổi Kiểu Dữ Liệu	11
8.3	Phân Biệt Hàm	11
9	Ưu Điểm và Đặc Điểm Của Dự Án	12
9.1	Độ Chính Xác Cao	12
9.2	Partial Pivoting	12
9.3	Xử Lý Nhiều Trường Hợp	12
9.4	Hiển Thị Rõ Ràng	12
10	Các Thuật Toán Không Được Sử Dụng	13
11	Kết Luận	14

1 Giới Thiệu Dự Án

Dự án này cài đặt các thuật toán cơ bản của đại số tuyến tính bằng Python, bao gồm:

- Khử Gauss (Gaussian Elimination)
- Tính định thức (Determinant Calculation)
- Tính ma trận nghịch đảo (Matrix Inverse)
- Tính hạng ma trận và cơ sở không gian (Rank and Basis Calculation)
- Các hàm hỗ trợ và định dạng hiển thị

Tất cả các tính toán đều sử dụng lớp `Fraction` từ thư viện chuẩn `fractions` để đảm bảo độ chính xác cao, tránh sai số do làm tròn số thực trong các phép toán.

2 Tổng Quan Cấu Trúc Dự Án

2.1 Danh Sách File

- `gaussian.py` - Hàm khử Gauss, thay thế ngược và các hàm hỗ trợ
- `determinant.py` - Tính định thức bằng khử Gauss
- `inverse.py` - Tính ma trận nghịch đảo bằng Gauss-Jordan
- `rank_basis.py` - Tính hạng và cơ sở không gian
- `part1_demo.ipynb` - Notebook minh họa với các bài kiểm thử
- `README.md` - Hướng dẫn sử dụng

3 Thuật Toán Khử Gauss (`gaussian.py`)

3.1 Mô Tả Chung

Tập `gaussian.py` chứa các hàm cốt lõi cho thuật toán khử Gauss:

3.1.1 Các Hàm Hỗ Trợ

`to_fraction(value)` Chuyển đổi giá trị (int, float, hoặc Fraction) thành kiểu `Fraction`:

```
1 def to_fraction(value):
2     if isinstance(value, Fraction):
3         return value
4     return Fraction(str(value))
```

`ding_dang_hien_thi(value)` Định dạng `Fraction` để hiển thị (ví dụ: " $\frac{3}{2}$ " hoặc "3" nếu mẫu = 1)

`in_ma_tran_dep(ma_tran)` In ma trận dưới dạng cấu trúc bảng gọn gàng, căn lề phải

3.1.2 Khử Ma Trận về Bậc Thang

Hàm `khu_ma_tran_ve_bac_thang`(`ma_tran_chuan_hoa`, `b_chuan_hoa`) thực hiện:

1. Duyệt qua từng cột từ trái sang phải
2. Tìm **pivot** - phần tử có giá trị tuyệt đối lớn nhất trong cột (Partial Pivoting)
3. Hoán đổi hàng nếu cần để đưa pivot lên vị trí hiện tại
4. Khử các phần tử dưới pivot bằng phép biến đổi hàng:

$$\text{Hàng } r := \text{Hàng } r - \frac{a_{r,c}}{a_{k,c}} \times \text{Hàng } k$$

Trả về:

- `ma_tran_sau_khu`: Ma trận dạng bậc thang
- `b_sau_khu`: Vector `b` sau khi áp dụng các phép biến đổi tương ứng
- `so_lan_doi_hang`: Số lần hoán đổi hàng
- `pivot_rows`: Danh sách chỉ số hàng chứa pivot
- `pivot_cols`: Danh sách chỉ số cột chứa pivot

3.1.3 Tìm Nghiệm Hệ Phương Trình

Hàm `tim_nghiem_he` xác định loại nghiệm:

1. **Hệ vô nghiệm**: Khi tồn tại hàng mà tất cả phần tử bên trái = 0 nhưng phần tử bên phải $\neq 0$
2. **Hệ có vô số nghiệm**: Khi số pivot < số cột (có biến tự do)
3. **Hệ có nghiệm duy nhất**: Khi số pivot = số cột

Với hệ có vô số nghiệm, hàm `nghiem_tong_quat_he_vo_so_nghiem` tính:

- Nghiệm riêng $\mathbf{x}_0$
- Các vector cơ sở tương ứng với các biến tự do
- Biểu thức tham số cho mỗi ẩn

3.1.4 Cách Sử Dụng

`gaussian_eliminate(A, b)` Giải hệ phương trình tuyến tính $A\mathbf{x} = \mathbf{b}$:

```

1 A = [[1, 2, 0, 2], [3, 5, -1, 6], [2, 4, 1, 2], [2, 0, -7, 11]]
2 b = [6, 17, 12, 7]
3 ma_tran_sau_khu, nghiem_he, so_lan_doi_hang = gaussian_eliminate(A, b)
4
5 if nghiem_he is None:
6     print("Vô nghiệm")
7 elif len(nghiem_he) > 0 and isinstance(nghiem_he[0], str):
8     print("Vô số nghiệm:", nghiem_he)
9 else:
10    print("Nghiệm duy nhất:", [float(v) for v in nghiem_he])

```

print_gaussian_eliminate(A, b) In kết quả định dạng đẹp ra màn hình

back_substitution(U, c) Giải hệ tam giác trên $U\mathbf{x} = \mathbf{c}$ bằng thay thế ngược:

$$x_i = \frac{c_i - \sum_{j=i+1}^n u_{ij}x_j}{u_{ii}}$$

4 Tính Định Thức (determinant.py)

4.1 Thuật Toán

Định thức được tính thông qua khử Gauss:

$$\det(A) = (-1)^s \times \prod_{i=1}^n a_{ii}$$

trong đó:

- a_{ii} là các phần tử trên đường chéo của ma trận dạng bậc thang
- s là số lần hoán đổi hàng

Chú ý: Nếu tồn tại hàng hoàn toàn bằng 0, định thức = 0.

4.2 Cách Sử Dụng

```

1 A = [[2, 3], [1, 5]]
2 det = determinant(A) # Tra về Fraction
3 print(f"Dinh thuc: {det}")
4
5 # Hoac in theo dinh dang dep
6 print_determinant(A)
```

4.3 Các Trường Hợp Kiểm Thử

- Ma trận 2×2 : $\det \begin{pmatrix} 2 & 3 \\ 1 & 5 \end{pmatrix} = 7$
- Ma trận 3×3 : $\det \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix} = -49$
- Ma trận phụ thuộc tuyến tính: $\det \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 2 \end{pmatrix} = 0$
- Ma trận đơn vị: $\det(I) = 1$
- Ma trận đường chéo: $\det = \text{tích các phần tử trên đường chéo}$

5 Tính Ma Trận Nghịch Đảo (inverse.py)

5.1 Thuật Toán Gauss-Jordan

Ma trận nghịch đảo được tính từ ma trận mở rộng $[A \mid I]$:

1. **Khử xuống:** Biến đổi $[A \mid I]$ thành $[U \mid X]$ (U là tam giác trên)
2. **Khử lên:** Tiếp tục khử để được $[I \mid A^{-1}]$

Điều kiện: Ma trận A phải là vuông và khả nghịch ($\det(A) \neq 0$).

5.2 Cách Sử Dụng

```

1 A = [[2, 3], [1, 5]]
2 A_inv = inverse(A) # Tra về danh sách các Fraction
3
4 if A_inv is not None:
5     print_inverse(A) # In A và A(-1)
6 else:
7     print("Ma trận không khả nghịch")

```

5.3 Ứng Dụng Kiểm Thử

Kiểm chứng: $A \times A^{-1} = I$

Ví dụ:

$$\begin{pmatrix} 2 & 3 \\ 1 & 5 \end{pmatrix}^{-1} = \begin{pmatrix} 5/7 & -3/7 \\ -1/7 & 2/7 \end{pmatrix}$$

6 Tính Hạng và Cơ Sở (rank_basis.py)

6.1 Định Nghĩa

Với ma trận A bất kỳ:

- **Hạng (Rank)**: Số lượng cột pivot = số lượng hàng độc lập tuyến tính
- **Cơ sở không gian cột** (Column Space): Các cột pivot của ma trận A gốc
- **Cơ sở không gian dòng** (Row Space): Các hàng khác 0 của ma trận dạng bậc thang
- **Cơ sở không gian nghiệm** (Null Space): Các vector cơ sở ứng với những cột không phải pivot

6.2 Thuật Toán

1. Khử ma trận A về dạng bậc thang
2. Xác định các cột pivot và hàng pivot
3. Trích xuất cơ sở từ các cột pivot
4. Tính không gian nghiệm từ các cột tự do

6.3 Cách Sử Dụng

```
1 A = [[1, 2, 3], [2, 4, 6], [0, 1, 2]]
2 rank, col_space, row_space, null_space = rank_and_basis(A)
3
4 print(f"Hang: {rank}")
5 print(f"Co so khong gian cot: {col_space}")
6 print(f"Co so khong gian dong: {row_space}")
7 print(f"Co so khong gian nghiem: {null_space}")
8
9 # Hoac in theo dinh dang dep
10 print_rank_and_basis(A)
```


7 Kiểm Chứng và Xác Nhận (part1_demo.ipynb)

7.1 Mục Đích

Notebook `part1_demo.ipynb` cung cấp các bài kiểm thử toàn diện để xác nhận tính chính xác của các thuật toán.

7.2 Nội Dung Chính

7.2.1 Test Gaussian Elimination

- **TEST 1:** Hệ có nghiệm duy nhất (kiểm chứng bằng NumPy)
- **TEST 2:** Hệ vô nghiệm
- **TEST 3:** Hệ có vô số nghiệm
- **TEST 4:** Trường hợp đặc biệt 1×1
- **TEST 5:** Trường hợp đặc biệt 2×4 (underdetermined)
- **TEST 6:** Partial pivoting (pivot đầu = 0)

7.2.2 Test Back Substitution

- Ma trận tam giác trên với nghiệm duy nhất
- Ma trận tam giác với hệ vô nghiệm
- Ma trận tam giác với hệ vô số nghiệm
- Trường hợp đặc biệt 1×1
- Xử lý lỗi với ma trận không vuông

7.2.3 Test Determinant

- Ma trận 2×2 , 3×3 , 4×4
- Ma trận phụ thuộc tuyến tính (định thức = 0)
- Ma trận đơn vị, ma trận đường chéo

7.2.4 Test Matrix Inverse

- Ma trận 2×2 , 3×3 , 4×4
- Ma trận đơn vị
- Ma trận không khả nghịch

7.2.5 Test Rank and Basis

- Ma trận vuông khả nghịch (full rank)
- Ma trận phụ thuộc tuyến tính (rank thiếu)
- Ma trận chữ nhật (underdetermined)
- Ma trận rank = 1
- Ma trận zero (rank = 0)

7.3 Kiểm Chứng bằng NumPy

Notebook cũng thực hiện kiểm chứng so sánh kết quả với các hàm tính toán có sẵn:

```
1 # Kiểm chứng nghiệm
2 my_x = gaussian_eliminate(A, b)[1]
3 np_x = np.linalg.solve(np.array(A), np.array(b))
4 flag1 = np.allclose(np.array([float(v) for v in my_x]), np_x)
5
6 # Kiểm chứng định thức
7 my_det = float(determinant(A))
8 np_det = float(np.linalg.det(np.array(A)))
9 flag2 = np.isclose(my_det, np_det)
10
11 # Kiểm chứng  $A * A^{-1} = I$ 
12 I_calc = A_np @ my_inv_np
13 flag3 = np.allclose(I_calc, np.eye(n))
14
15 # Kiểm chứng hạng
16 my_rank = rank_and_basis(A)[0]
17 np_rank = int(np.linalg.matrix_rank(np.array(A)))
18 flag4 = my_rank == np_rank
```

8 Hướng Dẫn Sử Dụng (README.md)

8.1 Giải Thích Hàm `gaussian_eliminate`

Hàm `gaussian_eliminate(A, b)` trả về tuple gồm 3 phần tử:

1. `[0]`: Ma trận dạng bậc thang (kiểu `Fraction`)
2. `[1]`: Nghiệm hệ
3. `[2]`: Số lần hoán đổi hàng (`int`)

8.1.1 Ý Nghĩa của Nghiệm Hệ

- `None`: Hệ vô nghiệm
- `list[Fraction]`: Hệ có nghiệm duy nhất
- `list[str]`: Hệ có vô số nghiệm (dạng tham số)

8.1.2 Ví Dụ Kiểm Tra Nhanh

```

1 if nghiệm_he is None:
2     print("Vô nghiệm")
3 elif len(nghiem_he) > 0 and isinstance(nghiem_he[0], str):
4     print("Vô số nghiệm:", nghiem_he)
5 else:
6     print("Nghiệm duy nhất:", nghiem_he)
```

8.2 Chuyển Đổi Kiểu Dữ Liệu

- **Không cần** chuyển đổi trước gọi hàm: hàm sẽ tự đổi `int/float` sang `Fraction`
- **Cần** đổi sang `float` khi:
 - In kết quả dạng thập phân
 - Đưa vào NumPy/vẽ đồ thị

Ví dụ:

```

1 x_float = [float(v) for v in nghiệm_he]
```

8.3 Phân Biệt Hàm

- `gaussian_eliminate(A, b)`: Lấy dữ liệu trả về để xử lý tiếp
- `print_gaussian_eliminate(A, b)`: In kết quả ra màn hình

9 Ưu Điểm và Đặc Điểm Của Dự Án

9.1 Độ Chính Xác Cao

- Sử dụng Fraction từ Python để tránh sai số làm tròn
- Tất cả phép toán được thực hiện trên số hữu tỷ
- Phù hợp cho các bài toán yêu cầu kết quả chính xác

9.2 Partial Pivoting

- Cách chọn pivot theo giá trị tuyệt đối lớn nhất
- Cải thiện ổn định số (numerical stability)
- Giảm hiện tượng mất mát độ chính xác trong phép tính

9.3 Xử Lý Nhiều Trường Hợp

- Hệ vô nghiệm, vô số nghiệm, và nghiệm duy nhất
- Ma trận vuông, hình chữ nhật, ma trận đặc biệt
- Hàng/cột toàn 0, ma trận suy biến

9.4 Hiển Thị Rõ Ràng

- Định dạng ma trận đẹp, dễ đọc
- Hiện thị Fraction theo dạng a/b dễ hiểu
- Đầu ra có cấu trúc và rõ ràng

10 Các Thuật Toán Không Được Sử Dụng

Theo yêu cầu của dự án, các hàm/cơ chế giải sẵn không được dùng trong việc triển khai:

- `numpy.linalg.solve`: Không dùng để giải hệ
- `numpy.linalg.inv`: Không dùng để tính ma trận nghịch đảo
- `scipy.linalg.qr`: Không dùng
- `scipy.linalg.lu`: Không dùng
- `sympy.linsolve`: Không dùng
- Echelon form/RREF từ SymPy: Không dùng

Chú ý: Các hàm thư viện trên chỉ được dùng cho **mục đích kiểm chứng** kết quả, không phải cài đặt thuật toán.

11 Kết Luận

Dự án này thành công triển khai các thuật toán cơ bản của đại số tuyến tính bằng Python:

- **Khử Gauss:** Giải hệ phương trình tuyến tính
- **Định thức:** Tính định thức thông qua khử Gauss
- **Ma trận nghịch đảo:** Sử dụng phương pháp Gauss-Jordan
- **Hạng và cơ sở:** Phân tích không gian vectơ

Tất cả các thuật toán:

1. Được triển khai từ đầu (không dùng thư viện có sẵn để tính)
2. Sử dụng `Fraction` để đảm bảo độ chính xác
3. Xử lý đầy đủ các trường hợp đặc biệt
4. Được kiểm chứng bằng NumPy

Dự án phù hợp cho mục đích giáo dục và tìm hiểu sâu về các thuật toán toán học.